

COMP 1324 Algorithmics

Graphs

Hikmat Farhat

Introduction

- A **graph** $G = (V, E)$ is a set of vertices V and a set of edges E connecting the vertices.
- Each element in E is a pair (v, w) with $v, w \in V$.
- If the pairs are **ordered** then the graph is **directed** (sometimes called **digraph**).
- if $(v, w) \in E$ then we say w is **adjacent** to v
- Usually we associate a **weight** (or **cost**) with each edge.
- A **path** is a sequence of vertices $w_1, \dots, w_n$ such that $(w_i, w_{i+1}) \in E$.
- the **length** of a path is the number of edges in it

- A path is said to be **simple** if all vertices, except possibly the first and last, are **distinct**.
- A **cycle** is a path such that $w_1 = w_n$.
- in an undirected graph we require that the edges be distinct to have a cycle.
- for example v, w, v should not be considered a cycle since (v, w) and (w, v) are the same edge.
- A graph is said to be **acyclic** if it contains no cycles.
- A graph in which from every vertex there is path to every other vertex is called **connected**.

- The degree of a vertex v , denoted $deg(v)$ is the number of edges incident on v .
- For a directed graph we have two degrees: the **indegree** and the **outdegree**.
- A useful property is that the sum of the degrees of all vertices is proportional to the number of edges. Undirected graph

$$\sum_{v \in V} deg(v) = 2 |E|$$

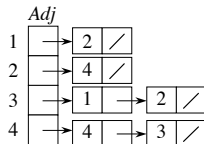
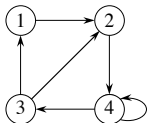
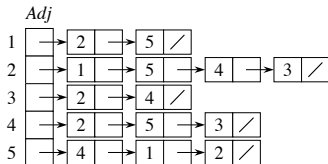
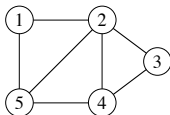
- For a directed graph.

$$\sum_{v \in V} indeg(v) = \sum_{v \in V} outdeg(v) = |E|$$

Graph representation

- There are essentially two ways to represent a graph
 - Adjacency matrix.
 - Adjacency list.
- Most of the time adjacency list is better since it is $O(|E| + |V|)$ in memory requirement.
- This is the preferred representation when the graph is sparse, $|E| \ll |V^2|$.
- The adjacency matrix is $O(|V^2|)$ in memory requirement and it is preferred when the graph is **dense**, $|E| \approx |V^2|$.
- In the adjacency matrix representation it is much faster to check whether two vertices are adjacent.

Examples



Matrix

	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0

	1	2	3	4
1	0	1	0	0
2	0	0	0	1
3	1	1	0	0
4	0	0	1	1

Breadth First Search

- As we will see later many algorithms depend on **breadth first search** (BFS).
- Given a graph $G = (V, E)$ and a **source** node s , BFS systematically "discovers" all vertices that can be reached from s .
- It is breadth first because all vertices at distance k from s are discovered **before** any vertex at distance $k + 1$ is discovered.
- In order not to visit a node more than once, BFS colors nodes with two different colors: **white** and **black**.
- A **white** node means it has not been discovered. **Black** means it has been discovered.

Breadth First Search

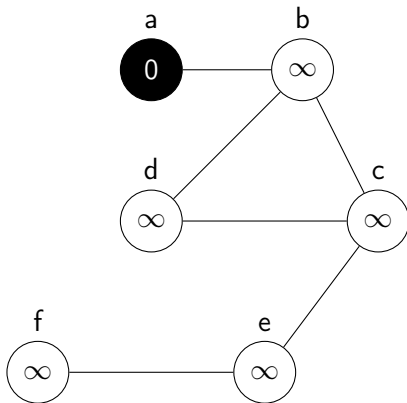
- The algorithm starts by coloring all nodes white except the source s is colored black.
- It then proceed with the discovery of all of s neighbors.
- Given a node v
 - $v.d$ is the distance (number of links) from s to v .
 - $adj[v]$ is the list of v 's neighbors.
 - $v.p$ is the predecessor of v in the path from s to v .

BFS Example

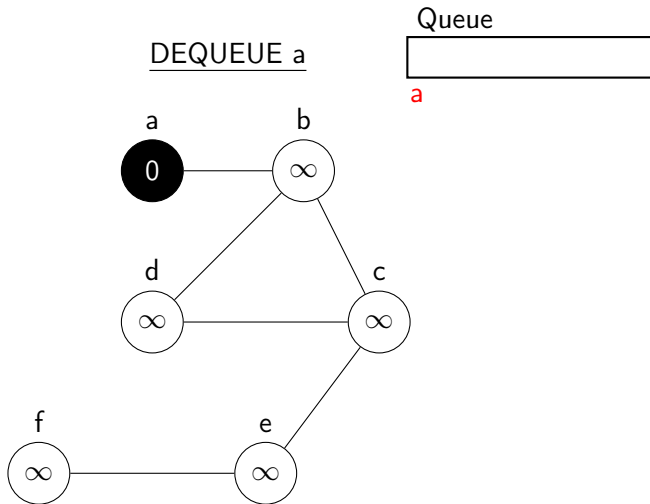
Initialise

Queue

a



BFS Example



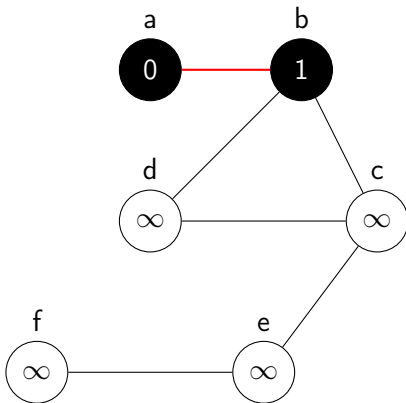
BFS Example

Visit neighbors of a

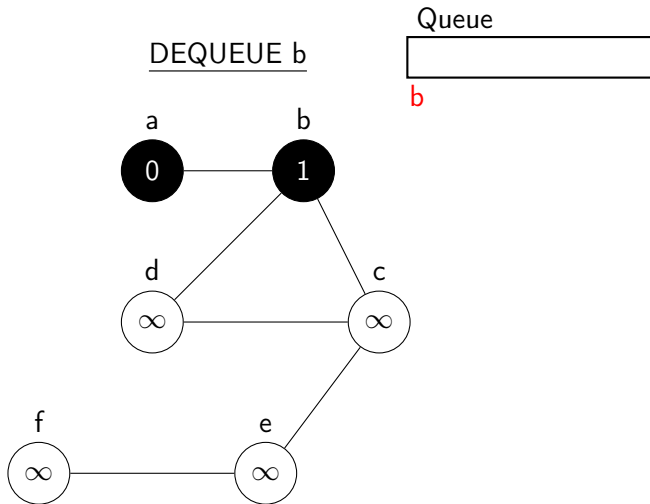
Queue

b

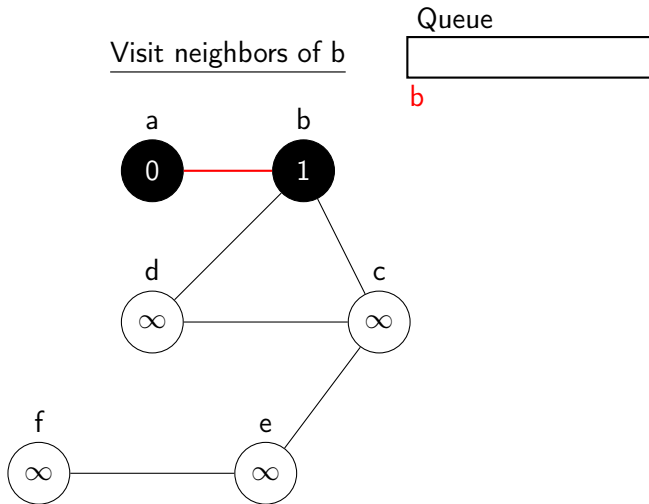
a



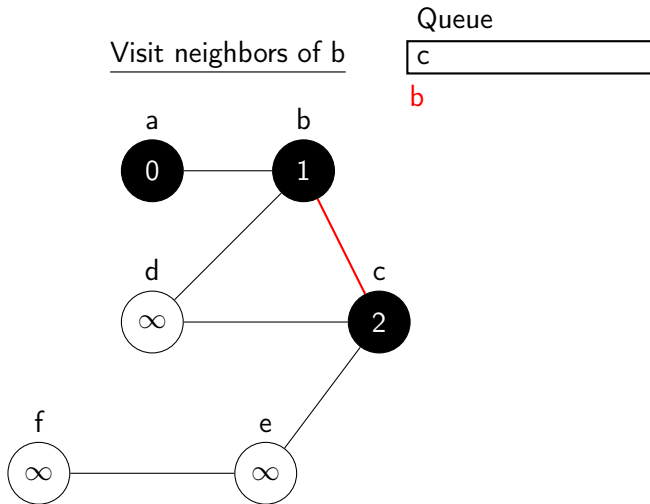
BFS Example



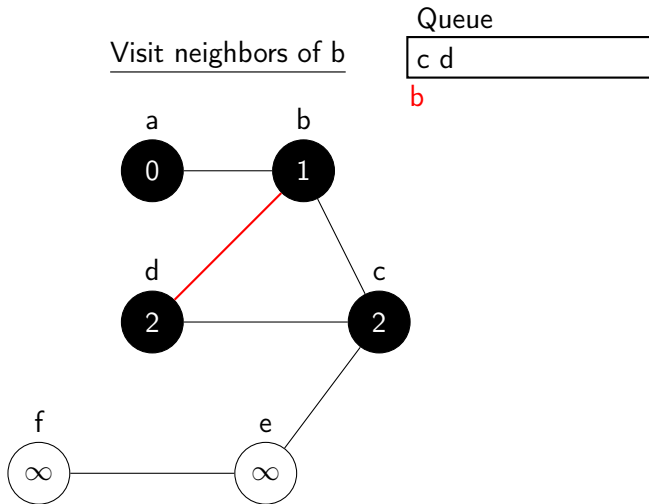
BFS Example



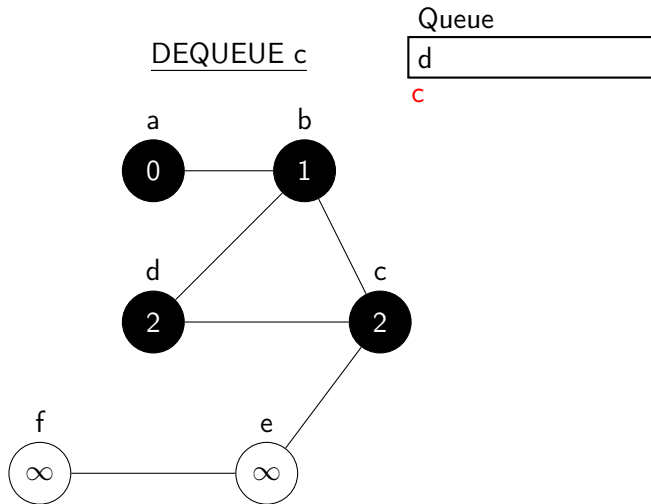
BFS Example



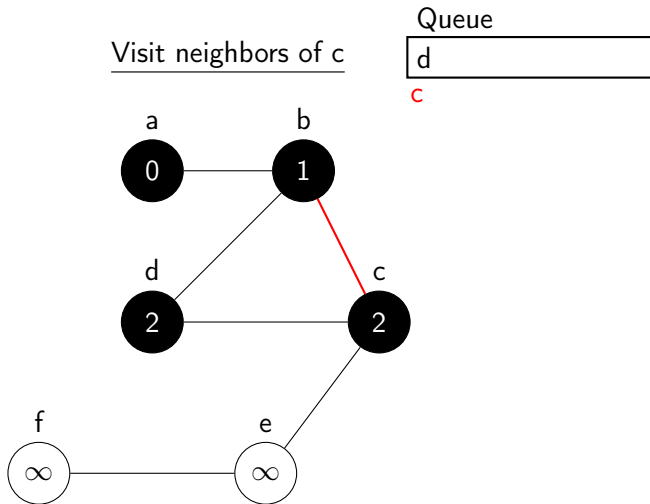
BFS Example



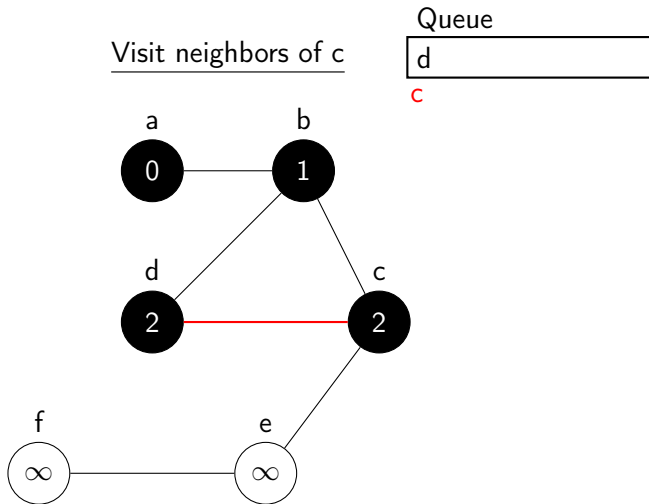
BFS Example



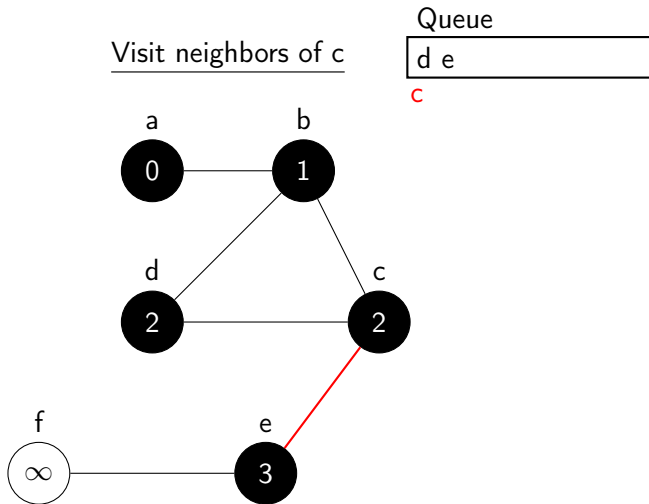
BFS Example



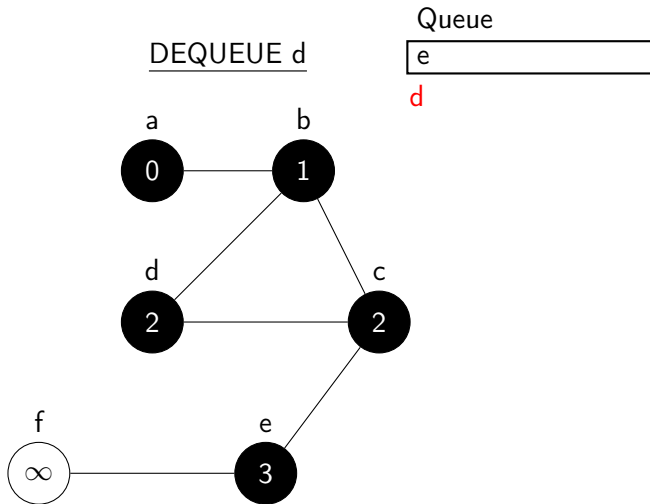
BFS Example



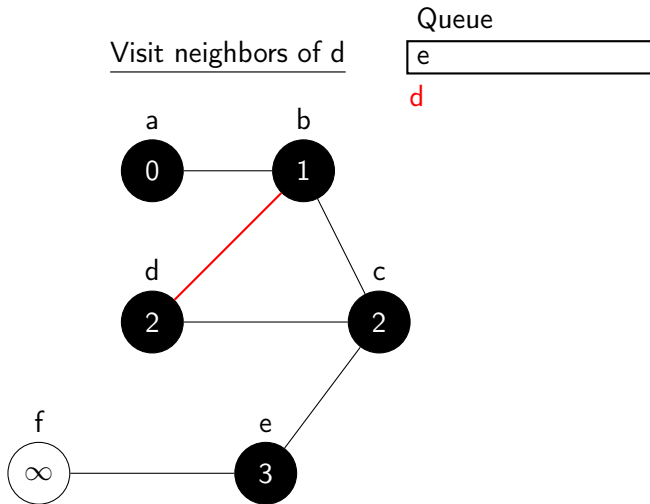
BFS Example



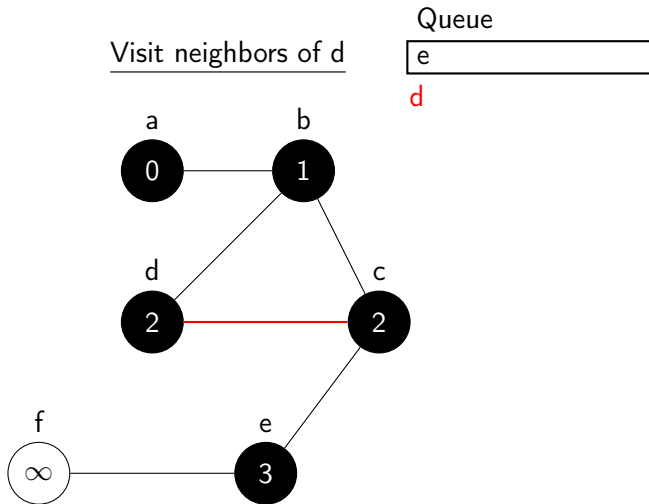
BFS Example



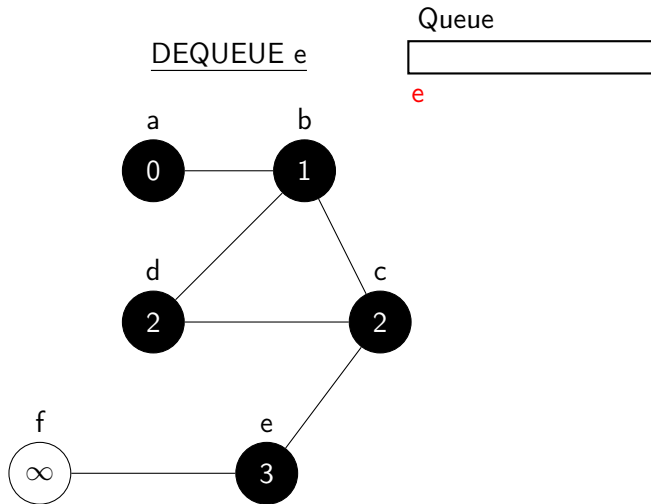
BFS Example



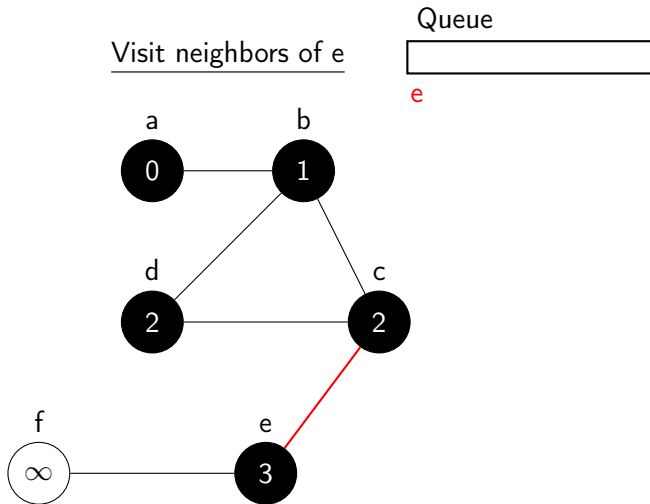
BFS Example



BFS Example



BFS Example



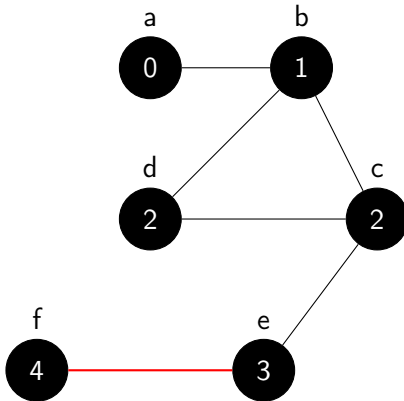
BFS Example

Visit neighbors of e

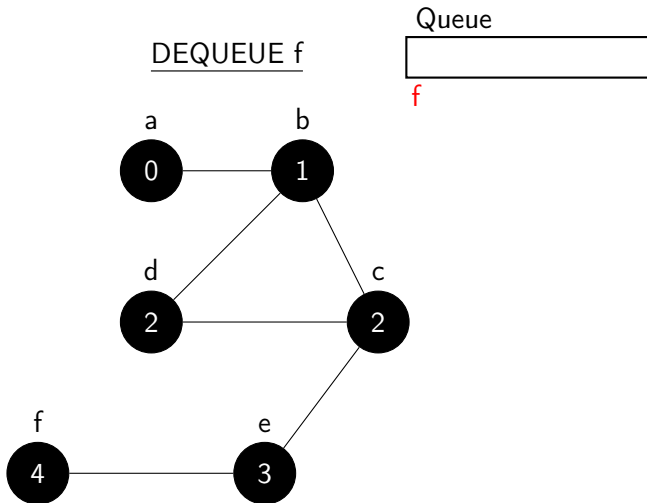
Queue

f

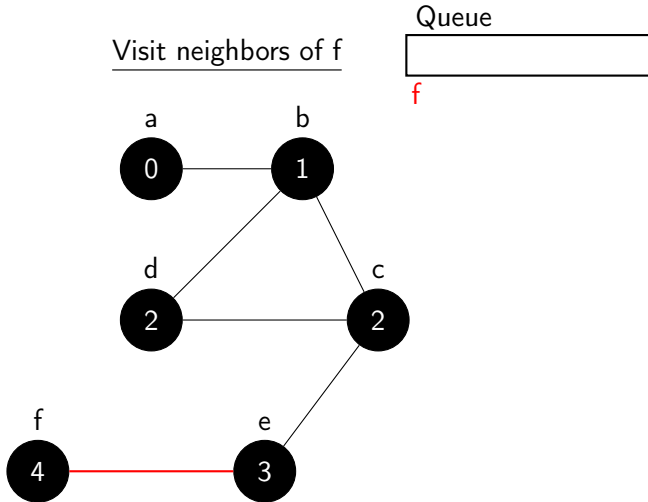
e



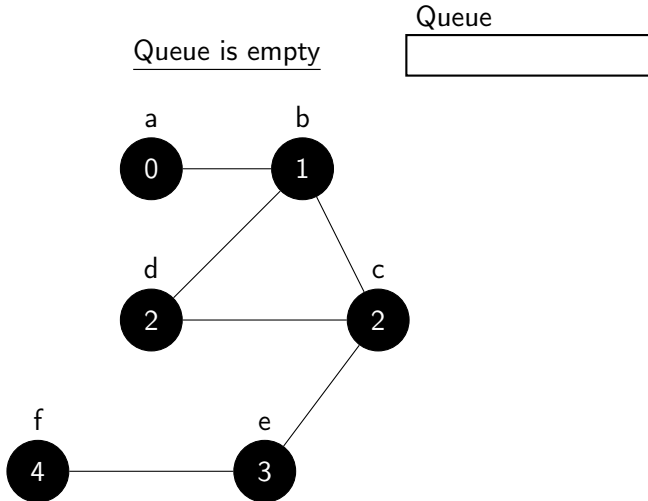
BFS Example



BFS Example



BFS Example



BFS Initialization

Given a graph G and source node s , BFS is initialised as follows:

Algorithm 1: BFS Initialization

```
1 function BFSinit( $G, s$ )
2   foreach  $v \in V - \{s\}$  do
3      $v.color \leftarrow WHITE$ 
4      $v.d \leftarrow \infty$ 
5      $v.p \leftarrow NULL$ 
6    $s.color \leftarrow BLACK$ 
7    $s.d \leftarrow 0$ 
8    $s.p \leftarrow NULL$ 
9    $Q \leftarrow \emptyset$ 
10  ENQUEUE( $Q, s$ )
```

BFS Pseudo Code

Algorithm 2: BFS main algorithm

input: A graph $G = (V, E)$ and a source node s

Result: Nodes reachable from s visited in BFS order. $v.d$ is the number of hops from s to v

```
1 function BFS( $G, s$ )
2   BFSinit( $G, s$ )
3   while  $Q \neq \emptyset$  do
4      $u \leftarrow \text{DEQUEUE}(Q)$ 
5     foreach  $v \in \text{Adj}[u]$  do
6       if  $v.\text{color} = \text{WHITE}$  then
7          $v.\text{color} \leftarrow \text{BLACK}$ 
8          $v.d \leftarrow u.d + 1$ 
9          $v.p \leftarrow u$ 
10        ENQUEUE( $Q, v$ )
```

BFS time complexity

- To analyze the complexity of BFS first we note that after initialization no vertex color is set to white.
- The above implies that each vertex is enqueued (and dequeued) *at most* once.
- Since the dequeue operations are $O(1)$ then the total cost of dequeue is $O(|V|)$.
- When a vertex is dequeued we scan the adjacency list and the sum of all adjacency list is just $O(|E|)$
- Therefore the total cost of BFS is $O(|V| + |E|)$.
- Why not Θ ?

BFS time complexity

- To analyze the complexity of BFS first we note that after initialization no vertex color is set to white.
- The above implies that each vertex is enqueued (and dequeued) *at most* once.
- Since the dequeue operations are $O(1)$ then the total cost of dequeue is $O(|V|)$.
- When a vertex is dequeued we scan the adjacency list and the sum of all adjacency list is just $O(|E|)$
- Therefore the total cost of BFS is $O(|V| + |E|)$.
- Why not Θ ? Some vertices may not be reachable from s .

BFS time complexity

Algorithm 2: BFS main algorithm

```
1 function BFS(G,s)
2   BFSinit(G,s)
3   while  $Q \neq \emptyset$  do
4      $u \leftarrow \text{DEQUEUE}(Q)$ 
5     foreach  $v \in \text{adj}[u]$  do
6       if  $v.\text{color} = \text{WHITE}$  then
7          $v.\text{color} \leftarrow \text{BLACK}$ 
8          $v.d \leftarrow u.d + 1$ 
9          $v.p \leftarrow u$ 
10    ENQUEUE(Q,v)
```

$\Theta(|V|)$
 $O(|V|)$ times

BFS time complexity

Algorithm 2: BFS main algorithm

1 function BFS(G, s) 2 BFSinit(G, s) 3 while $Q \neq \emptyset$ do 4 $u \leftarrow \text{DEQUEUE}(Q)$ 5 foreach $v \in \text{adj}[u]$ do 6 if $v.\text{color} = \text{WHITE}$ then 7 $v.\text{color} \leftarrow \text{BLACK}$ 8 $v.d \leftarrow u.d + 1$ 9 $v.p \leftarrow u$ 10 ENQUEUE(Q, v)	$\Theta(V)$ $O(V)$ times $\Theta(1)$
--	--

Algorithm 2: BFS main algorithm

```

1 function BFS(G,s)
2     BFSinit(G,s)  $\Theta(|V|)$ 
3     while  $Q \neq \emptyset$  do  $O(|V|)$  times
4          $u \leftarrow \text{DEQUEUE}(Q)$   $\Theta(1)$ 
5         foreach  $v \in \text{adj}[u]$  do  $|\text{adj}[u]|$  (adj list)
6             if  $v.\text{color} = \text{WHITE}$  then
7                  $v.\text{color} \leftarrow \text{BLACK}$ 
8                  $v.d \leftarrow u.d + 1$ 
9                  $v.p \leftarrow u$ 
10            ENQUEUE( $Q, v$ )

```

BFS time complexity

Algorithm 2: BFS main algorithm

1	function BFS(G, s)	
2	BFSinit(G, s)	$\Theta(V)$
3	while $Q \neq \emptyset$ do	$O(V)$ times
4	$u \leftarrow \text{DEQUEUE}(Q)$	$\Theta(1)$
5	foreach $v \in \text{adj}[u]$ do	$ \text{adj}[u] $ (adj list)
6	if $v.\text{color} = \text{WHITE}$ then	$\Theta(1)$
7	$v.\text{color} \leftarrow \text{BLACK}$	$\Theta(1)$
8	$v.d \leftarrow u.d + 1$	$\Theta(1)$
9	$v.p \leftarrow u$	$\Theta(1)$
10	ENQUEUE(Q, v)	$\Theta(1)$

BFS time complexity

Algorithm 2: BFS main algorithm

```
1 function BFS(G,s)
2   BFSInit(G,s)
3   while  $Q \neq \emptyset$  do
4      $u \leftarrow \text{DEQUEUE}(Q)$ 
5     foreach  $v \in \text{adj}[u]$  do
6       if  $v.\text{color} = \text{WHITE}$  then
7          $v.\text{color} \leftarrow \text{BLACK}$ 
8          $v.d \leftarrow u.d + 1$ 
9          $v.p \leftarrow u$ 
10        ENQUEUE(Q,v)
```

$O(\sum_{u \in V} |\text{adj}[u]|) = O(|E|)$

$O(|V|)$ times

$\Theta(1)$

$|\text{adj}[u]|$ (adj list)

$\Theta(1)$

$\Theta(1)$

$\Theta(1)$

$\Theta(1)$

$\Theta(1)$

BFS time complexity

Algorithm 2: BFS main algorithm

```

1 function BFS( $G, s$ )
2   BFSinit( $G, s$ )  $O(|V|)$ 
3   while  $Q \neq \emptyset$  do
4      $u \leftarrow \text{DEQUEUE}(Q)$   $O(|V|)$ 
5     foreach  $v \in \text{adj}[u]$  do
6       if  $v.\text{color} = \text{WHITE}$  then
7          $v.\text{color} \leftarrow \text{BLACK}$ 
8          $v.d \leftarrow u.d + 1$ 
9          $v.p \leftarrow u$ 
10        ENQUEUE( $Q, v$ )
  
```

}

$O(|E|)$

Total time complexity of BFS is $O(|V| + |E|)$.

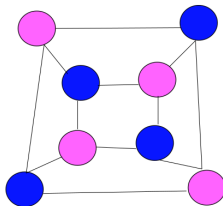
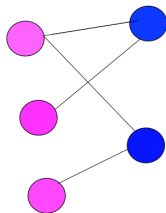
Shortest path length (number of hops)

- Given a graph $G = (V, E)$ and a source node $s \in V$. We define the **shortest-length** distance $\delta(s, v)$ from s to $v \in V$ to be the minimum **number of edges** in any path from s to v .
- When BFS terminates, it discovers every vertex $v \in V$ reachable from a source s and $v.d = \delta(s, v)$,
- Note that the shortest length path from s to v is **composed** of the shortest length path from s to $v.p$ **followed** by the edge $(v.p, v)$.
- The above observation allows us to determine not only the length $\delta(s, v)$ but also the exact path by iterating backwards over $v.p$.
- Also, all vertices at distance k from s are discovered (visited) before any vertex at distance $k + 1$.

Bipartite Graphs

- A graph $G = \langle V, E \rangle$ is a bipartite graph if V can be partitioned into two sets V_1 and V_2 such that
- $(u, v) \in E \Rightarrow u \in V_1$ and $v \in V_2$ or $u \in V_2$ and $v \in V_1$.
- A bipartite graph can be 2-colored. In other words,
 - Using only two colors, one can assign a color to each vertex
 - such that no two adjacent vertices have the same color.

Example bipartite graphs



Detecting bipartite graphs using BFS

- One can detect if a graph is bipartite using a small modification of BFS
- Instead of coloring a node black when it is "discovered" we either color it red or blue.
- The algorithm starts by coloring the source node red and all other nodes white exactly as it was done in BFS.
- While discovering the neighbors of a node u we color them with the opposite color of u .
- Let $\neg Blue = Red$ and $\neg Red = Blue$.

Detecting bipartite graphs using BFS

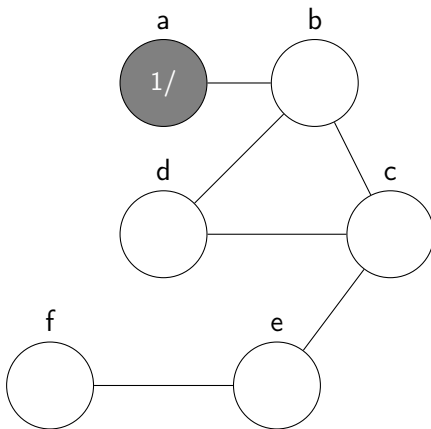
```
1 function IsBipartite(G,s)
2   while  $Q \neq \emptyset$  do
3      $u \leftarrow \text{DEQUEUE}(Q)$ 
4     foreach  $v \in \text{Adj}[u]$  do
5       if  $v.\text{color} = \text{WHITE}$  then
6          $v.\text{color} \leftarrow \neg u.\text{color}$ 
7          $v.d \leftarrow u.d + 1$ 
8          $v.p \leftarrow u$ 
9          $\text{ENQUEUE}(Q, v)$ 
10      else if  $u.\text{color} = v.\text{color}$  then
11        return false
12  return true
```

Depth First Search

- In a **depth first search** DFS edges are explored out of the most recently discovered node. As the name implies we go "deeper" whenever it is possible.
- When all the neighbors of a node v are discovered we "backtrack" to the predecessor of v and explore other nodes.
- When the algorithm is done with a certain node, it records the **discovery time** and **finishing time**

DFS Example

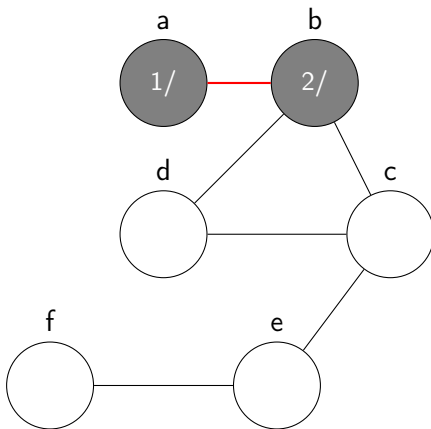
"Pending"



a

DFS Example

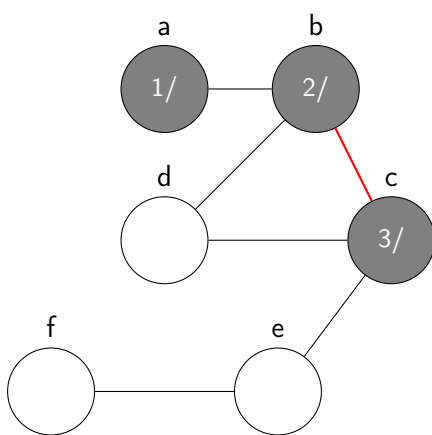
"Pending"



b
a

DFS Example

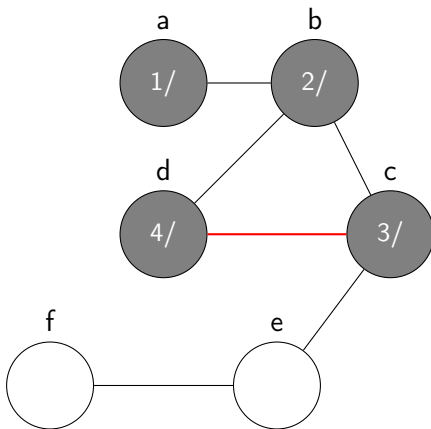
"Pending"



DFS Example

"Pending"

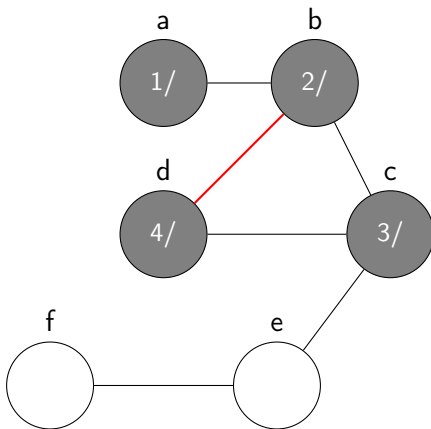
d
c
b
a



DFS Example

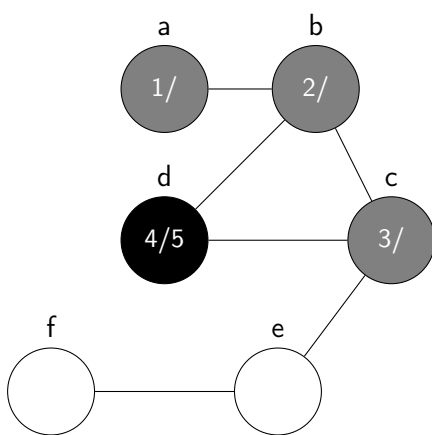
"Pending"

d
c
b
a



DFS Example

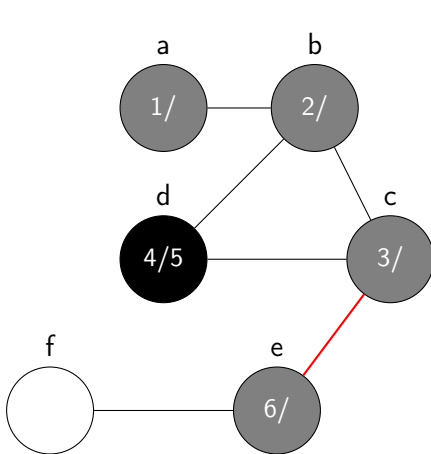
"Pending"



c
b
a

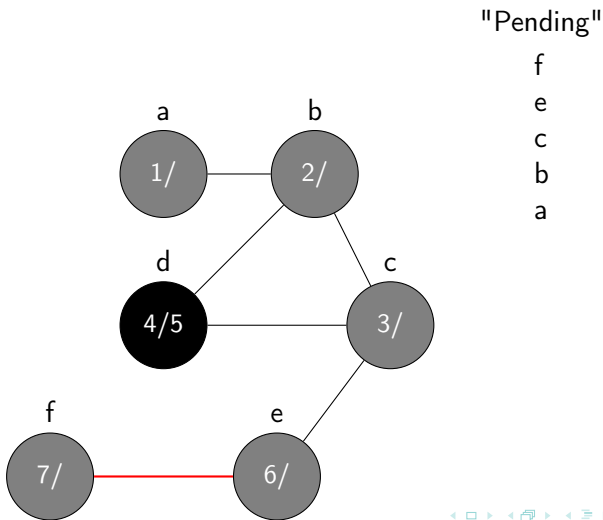
DFS Example

"Pending"



e
c
b
a

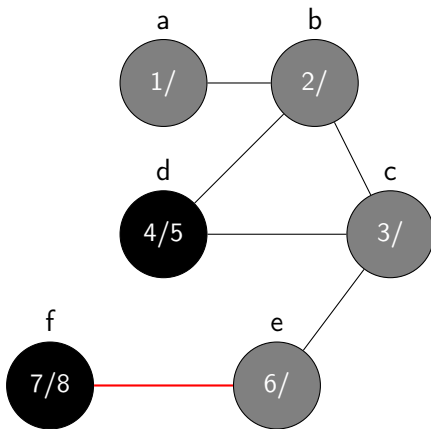
DFS Example



DFS Example

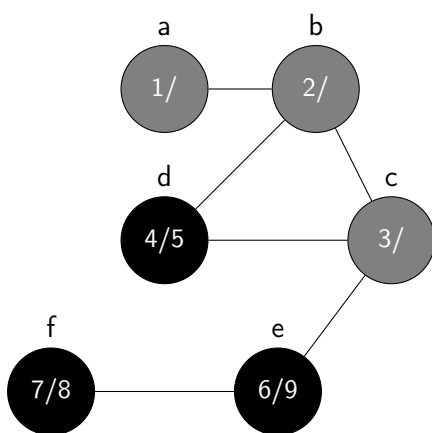
"Pending"

e
c
b
a



DFS Example

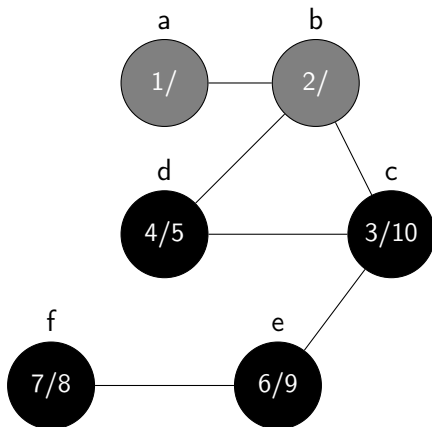
"Pending"



c
b
a

DFS Example

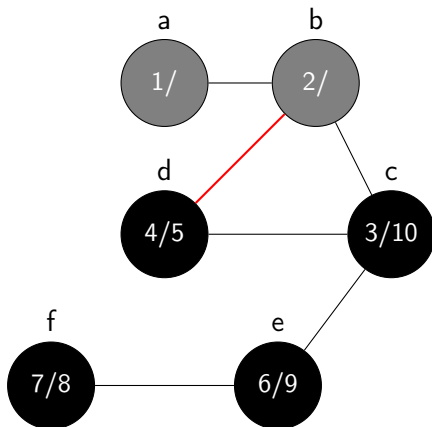
"Pending"



b
a

DFS Example

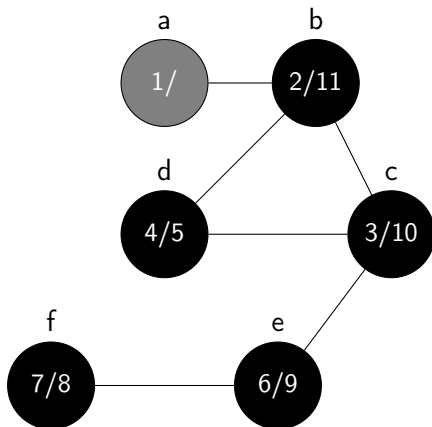
"Pending"



b
a

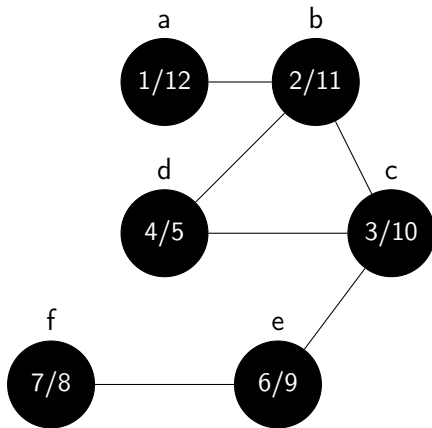
DFS Example

"Pending"



a

DFS Example



DFS-VISIT Pseudo Code

Algorithm 4: DFS-VISIT Algorithm

input: Adjacency list adj , node u

Result: All nodes reachable from u are visited in DFS order

```
1 function DFS-VISIT( $adj, u$ )
2    $u.color \leftarrow GRAY$ 
3    $time \leftarrow time + 1$ 
4    $u.d \leftarrow time$ 
5   foreach  $v \in adj[u]$  do
6     if  $v.color = WHITE$  then
7        $v.p \leftarrow u$ 
8       DFS-VISIT( $v$ )
9    $u.color \leftarrow BLACK$ 
10   $times \leftarrow time + 1$ 
11   $u.f \leftarrow time$ 
```

Complexity

- The initialization to WHITE is $\Theta(|V|)$
- Each time DFS-VISIT is called **exactly once** for each node because it is called on WHITE nodes only.
- The cost of $\text{DFS-VISIT}(v)$ is $\Theta(|\text{adj}[v]|)$ (using adjacency list representation)
- Thus the cost of all calls to DFS-VISIT is

$$\sum_{v \in V} |\text{adj}[v]| = \Theta(|E|)$$

- Therefore the total cost is

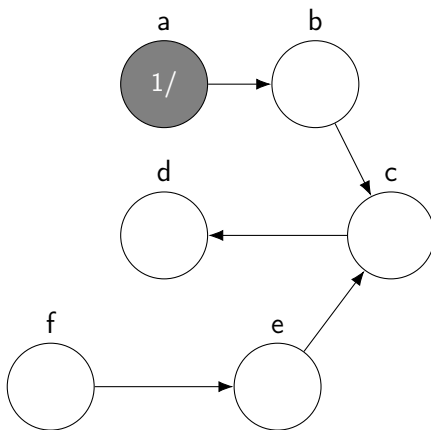
$$\Theta(|E| + |V|)$$

Topological Sorting (ordering)

- Topological sorting is a linear ordering of the nodes of a **directed** graph $G = (V, E)$ such that for every edge (u, v) from node u to node v , u comes before v in the ordering.
- An example is the scheduling of tasks based on their dependencies.
- Say a task p depends on task q and cannot start until q finishes. This would be represented by an edge (q, p) .
- We can implement an efficient topological sorting using DFS as follows
 - 1 Call DFS on the graph.
 - 2 Every time the processing of a node is finished it is to the **front** of a linked list
 - 3 When the algorithm terminates, the resulting list is the topological sorting.

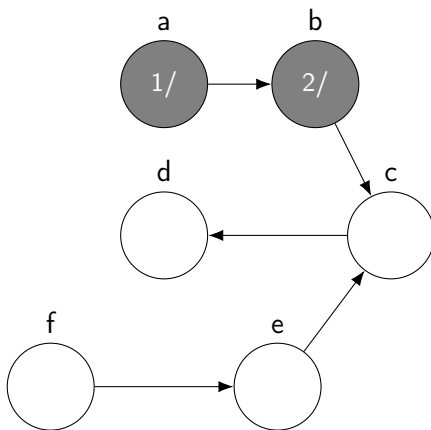
Topological Sorting Example

"Linked List"



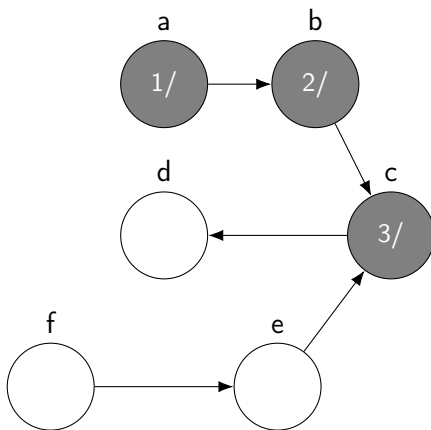
Topological Sorting Example

"Linked List"



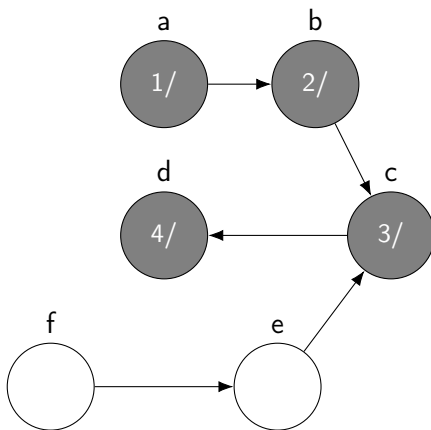
Topological Sorting Example

"Linked List"



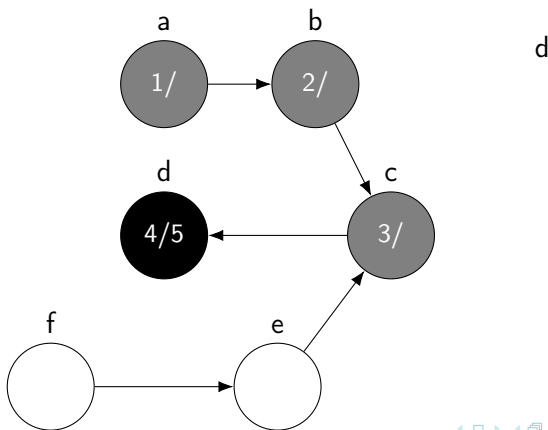
Topological Sorting Example

"Linked List"



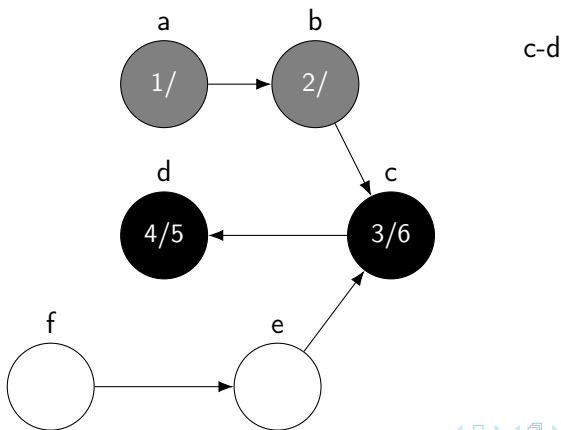
Topological Sorting Example

"Linked List"



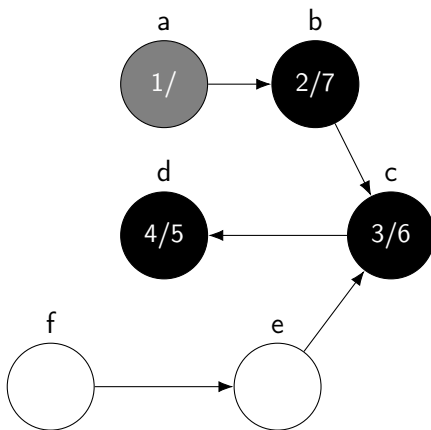
Topological Sorting Example

"Linked List"



Topological Sorting Example

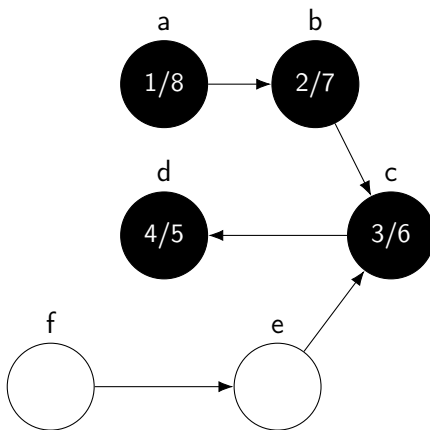
"Linked List"



b-c-d

Topological Sorting Example

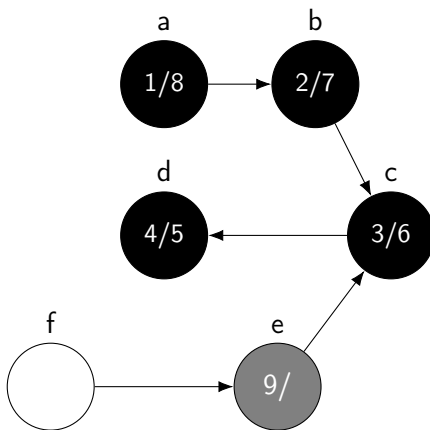
"Linked List"



a-b-c-d

Topological Sorting Example

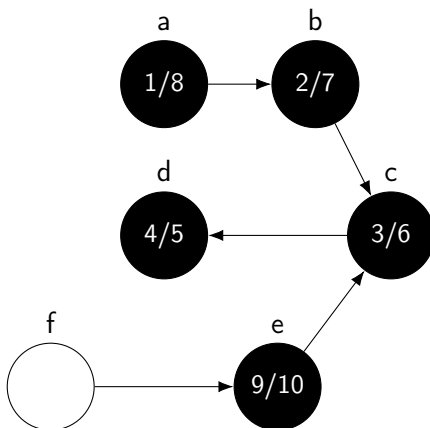
"Linked List"



a-b-c-d

Topological Sorting Example

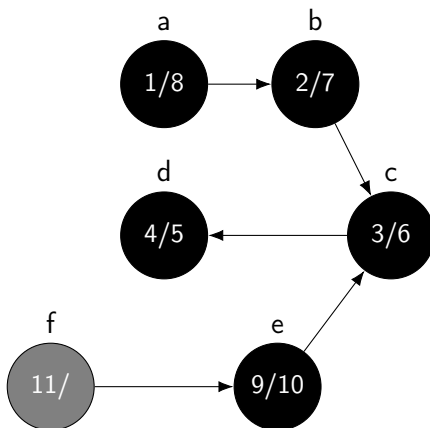
"Linked List"



e-a-b-c-d

Topological Sorting Example

"Linked List"

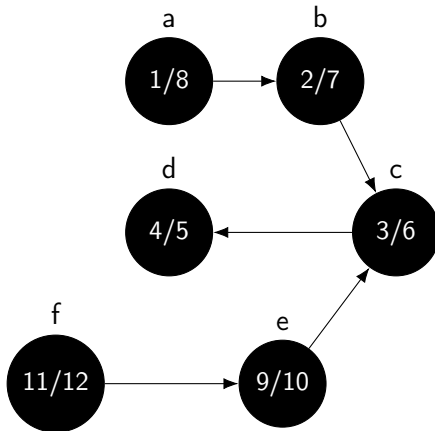


e-a-b-c-d

Topological Sorting Example

"Linked List"

f-e-a-b-c-d



Connected Components

- A **connected component** of an undirected graph is a subgraph in which there is a path between every pair of vertices.
- Since the graph is **undirected** if there is a path from a to b then there is a path from b to a .
- so in the special case of an undirected graph the connected components can be found by a simple DFS.
- Recall that $\text{DFS-VISIT}(v)$ will visit all vertices reachable from v .

```
1 foreach  $v \in V$  do
2   if  $v.\text{color} = \text{WHITE}$  then
3     a new component is found;
4     DFS-VISIT( $\text{adj}, v$ );
```

Strongly Connected Components

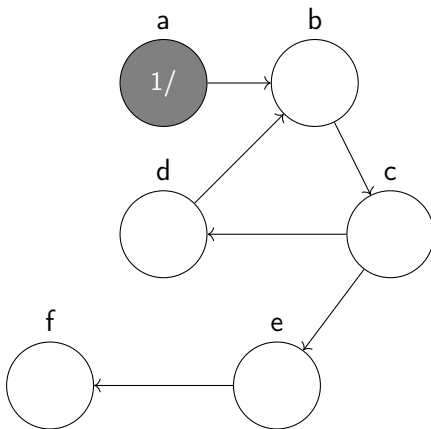
- A **directed** graph is said to be **strongly connected** if there is a path from every vertex to every other vertex.
 - A **strongly connected component** is a subgraph that is strongly connected.
 - One simple algorithm to find all strongly connected components is Kosaraju's algorithm.
- 1 call DFS to compute finishing time for all nodes.
 - 2 create G^T .
 - 3 call DFS in order of decreasing finishing time on G^T .

Kosaraju Algorithm

```
1 foreach  $v \in V$  do
2   if  $v.color = WHITE$  then
3     DFS-VISIT( $v$ )
4 Reverse all the edges of  $G$  and reset all colors
5 foreach  $v \in V$  in decreasing finish time do
6   if  $v.color = WHITE$  then
7     /*  $v$  and all vertices reachable from  $v$  form a
       strongly connected component */
8     DFS-VISIT( $v$ )
```

Run DFS first time

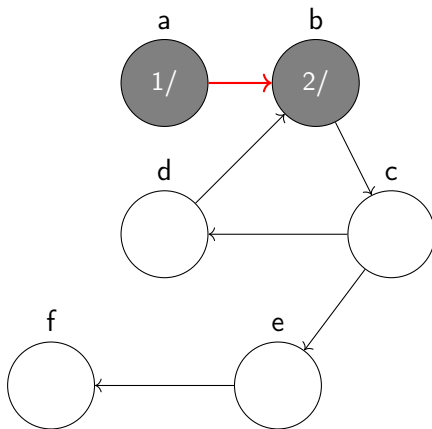
"Pending"



a

Run DFS first time

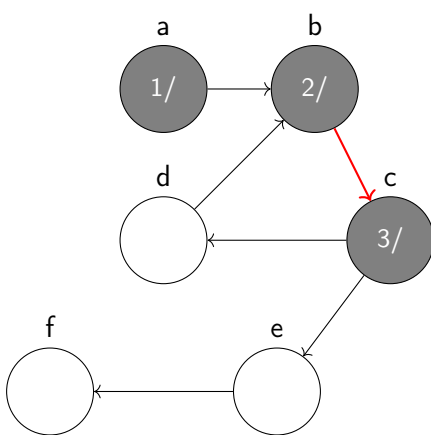
"Pending"



b
a

Run DFS first time

"Pending"

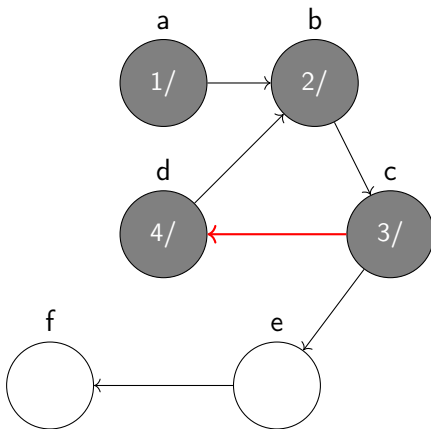


c
b
a

Run DFS first time

"Pending"

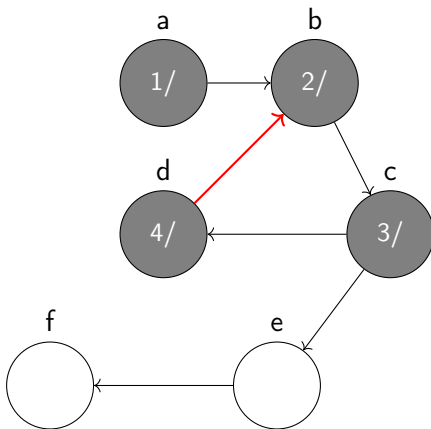
d
c
b
a



Run DFS first time

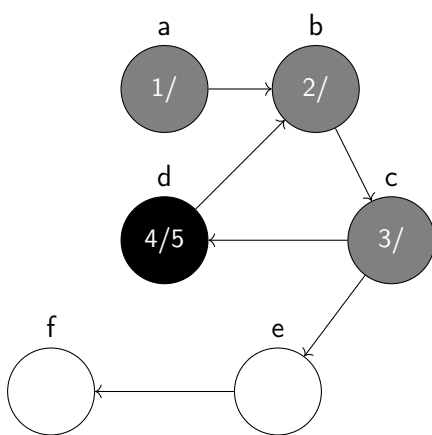
"Pending"

d
c
b
a



Run DFS first time

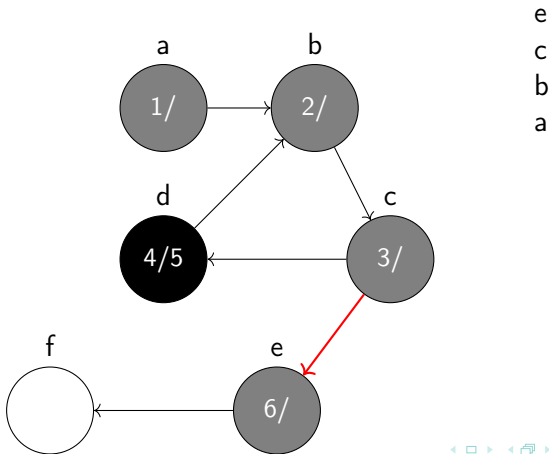
"Pending"



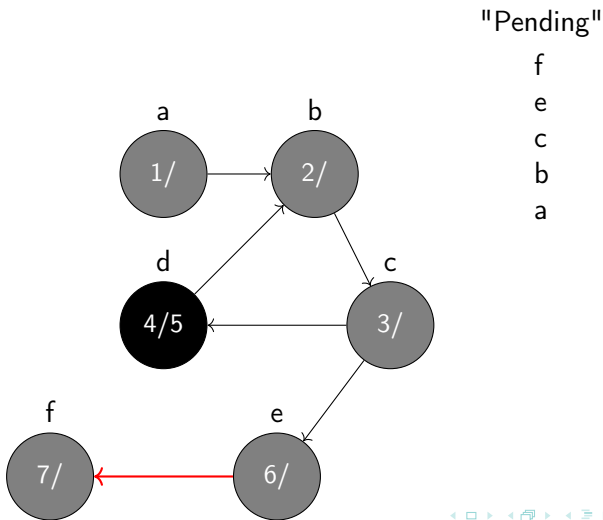
c
b
a

Run DFS first time

"Pending"



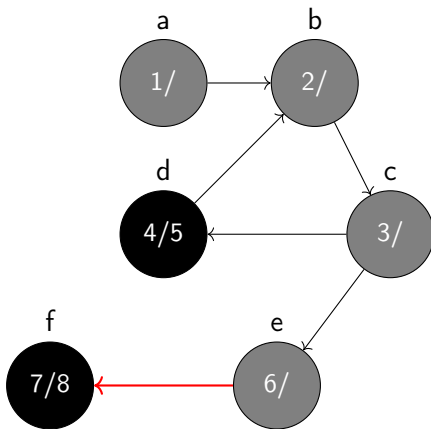
Run DFS first time



Run DFS first time

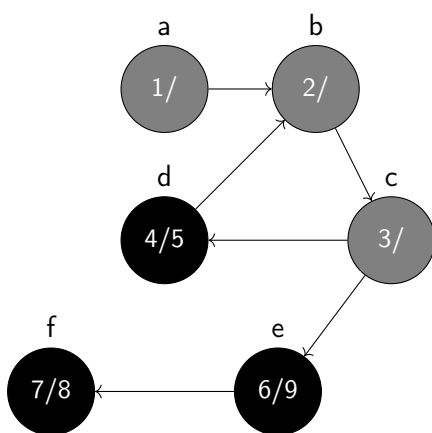
"Pending"

e
c
b
a



Run DFS first time

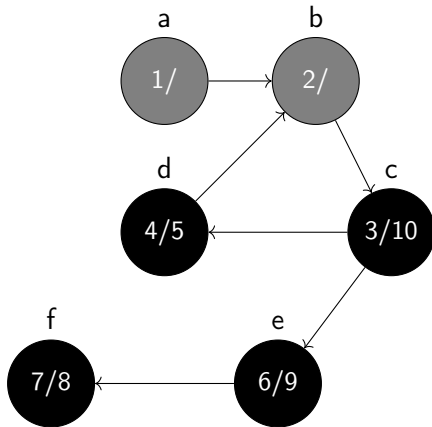
"Pending"



c
b
a

Run DFS first time

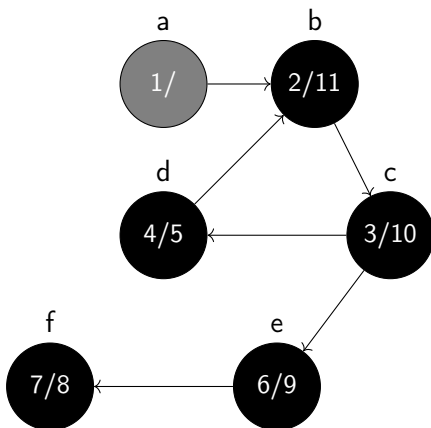
"Pending"



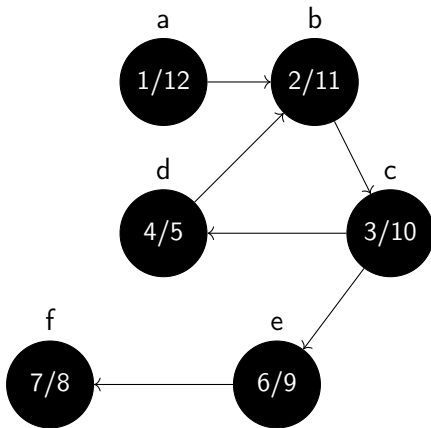
b
a

Run DFS first time

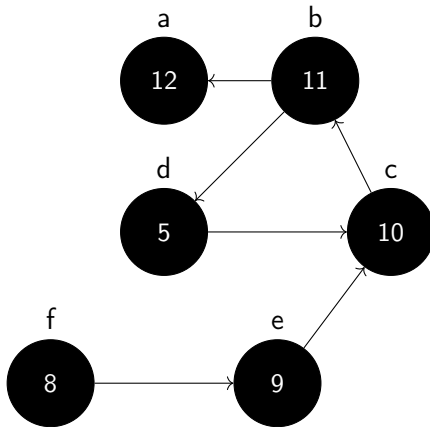
"Pending"



Run DFS first time

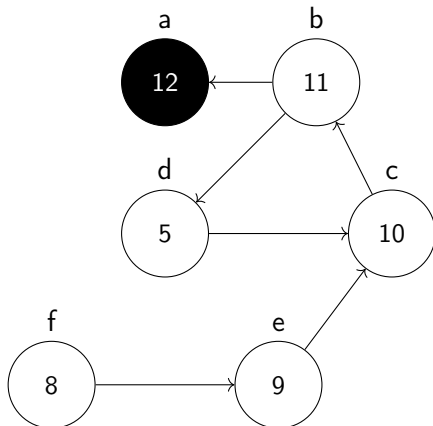


Reverse edges



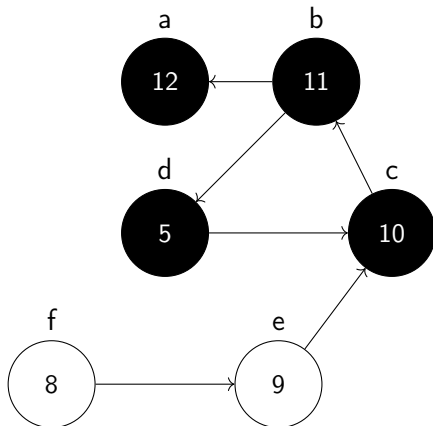
Clear colors and run DFS again

start with visiting a



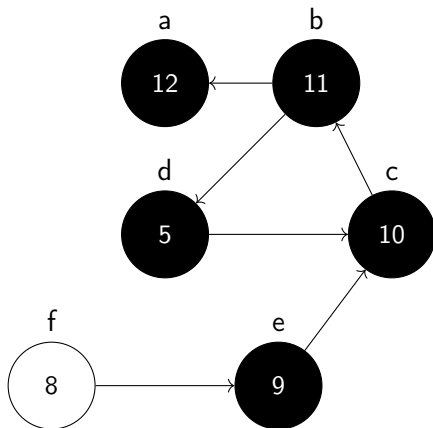
Clear colors and run DFS again

visit b discover d,c



Clear colors and run DFS again

visit e



Clear colors and run DFS again

visit f

